CS 2143 · OBJECT-ORIENTED PROGRAMMING

Object Lifetime & Ownership — Who Cleans Up?

Review worksheet — destructors, the Rule of Three / Five / Zero, move semantics, RAII, smart pointers, ownership — builds on Worksheets 03, 04 & 06

Name _____ Date _____ Section _____

part 1 When Do Objects Die?

builds on W05 construction order

Every object has a **lifetime**: it starts when its constructor finishes and ends when its destructor runs. *When* the destructor runs depends on how the object was created:

KIND OF OBJECT	CREATED BY	DESTROYED WHEN…
Automatic (local)	`Tracer a("A");`	its enclosing `{ }` block ends, in *reverse* order of creation
Parameter (by value)	calling `Visit(b)`	the function returns
Dynamic (heap)	`new Tracer("C")`	someone calls `delete`. If nobody does, never (a leak).

This class prints a line every time one of its objects is born, copied, or dies. A copy gets the original's name plus a `'`.

```
class Tracer {
public:
    Tracer(const string& name)
        : name_(name) {
        cout << "+" << name_ << " ";
    }
    Tracer(const Tracer& other)
        : name_(other.name_ + "'") {
        cout << "copy:" << name_ << " ";
    }
    ~Tracer() {
        cout << "-" << name_ << " ";
    }
private:
    string name_;
};

void Visit(Tracer t) {
    cout << "visit ";
}

int main() {
    Tracer a("A");
    {
        Tracer b("B");
        Tracer* c = new Tracer("C");
        Visit(b);
    }
    Tracer d("D");
    cout << "end ";
}
```

QUESTION 1 — PREDICT THE OUTPUT

Write the complete output, in order. (There are 11 items.)

_____

_____

QUESTION 2 — SHORT ANSWER

1. Which object is created but **never** destroyed? _____ Why?

   _____

2. The pointer `c` itself *is* destroyed at the closing `}`. Why doesn't that destroy the `Tracer` it points to?

   _____

Object Lifetime & Ownership — Who Cleans Up?

Review worksheet — destructors, the Rule of Three / Five / Zero, move semantics, RAII, smart pointers, ownership — builds on Worksheets 03, 04 & 06

part 2 The Big Three

finishes the W03/W04 preview

If you don't write these, the compiler does. Its versions copy or destroy each member in turn.

SPECIAL MEMBER	SIGNATURE	RUNS WHEN…
Destructor	`~IntStack()`	an object's lifetime ends
Copy constructor	`IntStack(const IntStack& other)`	a **new** object is made as a copy
Copy assignment	`IntStack& operator=(const IntStack& other)`	an **existing** object is overwritten

For an `int*` member, "copy the member" copies the *address*: the shallow copy from W04. Here is the W01 stack with only a constructor and destructor:

```
class IntStack {
public:
    explicit IntStack(int capacity)
        : data_(new int[capacity]), size_(0), capacity_(capacity) {
    }
    ~IntStack() {
        delete[] data_;
    }
    // ... Push, Pop, Top ...
private:
    int* data_;
    int size_;
    int capacity_;
};

void Show(IntStack s);    void Peek(const IntStack& s);    // Show copies, Peek borrows
```

QUESTION 3 — WHICH SPECIAL MEMBER RUNS?

For each line, write **ctor**, **copy ctor**, **copy assign**, or **none**.

LINE	RUNS	LINE	RUNS
`IntStack a(10);`		`Show(a);`	
`IntStack b = a;`		`Peek(a);`	
`IntStack c(a);`		`IntStack* p = &a;`	
`c = b;`		`a = a;`	

`IntStack b = a;` has an `=` in it. Why isn't it an assignment?

QUESTION 4 — WHAT GOES WRONG?

With only the class above (no copy constructor, no `operator=`), describe exactly what goes wrong at the end of `main()` for each line.

LINE	PROBLEM
`IntStack b = a;`	
`c = b;` (c had its own array)	

The Rule of Three: if a class needs a hand-written _____, _____, or _____, it almost certainly needs all three.

Object Lifetime & Ownership — Who Cleans Up?

Review worksheet — destructors, the Rule of Three / Five / Zero, move semantics, RAII, smart pointers, ownership — builds on Worksheets 03, 04 & 06

part 3 Writing Copy Assignment

You wrote a deep-copy constructor in W04. Copy assignment is harder, because the object on the left **already owns an array**. Each version below has exactly one serious bug.

QUESTION 5 · VERSION 1 — FIND THE BUG

```
IntStack& operator=(const IntStack& other) {
    data_ = other.data_;
    size_ = other.size_;
    capacity_ = other.capacity_;
    return *this;
}
```

Bug:

QUESTION 5 · VERSION 2 — FIND THE BUG

```
IntStack& operator=(const IntStack& other) {
    delete[] data_;
    data_ = new int[other.capacity_];
    for (int i = 0; i < other.size_; ++i) {
        data_[i] = other.data_[i];
    }
    size_ = other.size_;
    capacity_ = other.capacity_;
    return *this;
}
```

Bug (hint: `a = a;`):

QUESTION 5 · VERSION 3 — FIND THE BUG

```
void operator=(const IntStack& other) {
    if (this == &other) {
        return;
    }
    // ... correct deep copy ...
}
```

This one copies correctly. Which line no longer compiles, `a = b;` or `a = b = c;`? _____ Why?

QUESTION 6 — WRITE IT CORRECTLY — OPERATOR=

Write a correct copy assignment operator for `IntStack`. Handle self-assignment, don't leak the old array, deep copy, and return the right thing.

Object Lifetime & Ownership — Who Cleans Up?

Review worksheet — destructors, the Rule of Three / Five / Zero, move semantics, RAII, smart pointers, ownership — builds on Worksheets 03, 04 & 06

part 4 Move Semantics

C++11

Sometimes the object you're copying is about to be thrown away, like a temporary returned from a function. A deep copy is wasted work: you build a new array, copy every element, and then the original deletes its array a moment later. **Moving** lets the new object *steal* the old object's resources instead.

EXPRESSION	KIND	MEANING
`a`, `names[0]`, `*p`	**lvalue**	has a name or address and will still be around. Must copy.
`IntStack(5)`, `MakeStack()`, `a + b`	**rvalue**	a temporary that's about to disappear. Safe to steal from.
`std::move(a)`	rvalue	"I promise I'm done with `a`. Treat it as a temporary." It's only a *cast*. It moves nothing by itself.

A **move constructor** takes an *rvalue reference*, `IntStack&&`. It copies the pointer, then leaves the source empty but still **safe to destroy**:

```
IntStack(IntStack&& other) noexcept
    : data_(other.data_), size_(other.size_), capacity_(other.capacity_) {
    // ??? -- what must happen to other here?
}
```

QUESTION 7 — FINISH THE MOVE CONSTRUCTOR

Write the body.

1. What happens when `other`'s destructor runs if you leave the body empty?

   _____

2. A move does a fixed amount of work. A deep copy of an `IntStack` holding *n* items does how much?

   _____

3. `noexcept` promises the move can't throw. Why is a move that only copies pointers and integers safe to promise that, when a deep copy (which calls `new`) isn't?

   _____

QUESTION 8 — TRUE / FALSE

`std::move(a)` moves `a`'s data somewhere. _____

After `IntStack d = std::move(b);`, you may still call `b`'s destructor or assign `b` a new value. _____

After a move, reading `b.Top()` is safe and returns the old top. _____

Moving makes sense for an `int` member, which is cheaper to move than to copy. _____

`std::string` and `std::vector` already have move constructors. _____

Object Lifetime & Ownership — Who Cleans Up?

Review worksheet — destructors, the Rule of Three / Five / Zero, move semantics, RAII, smart pointers, ownership — builds on Worksheets 03, 04 & 06

part 4 Move Semantics (continued)

Give `Tracer` from Part 1 a move constructor. A moved-to object gets the name plus `*`, and the moved-from object is renamed `~`.

```
Tracer(Tracer&& other) noexcept
    : name_(other.name_ + "*") {
    other.name_ = "~";
    cout << "move:" << name_ << " ";
}

Tracer MakeTracer() {
    return Tracer("T");
}

vector<Tracer> v;
v.reserve(4);        // room for 4, so the vector never has to grow
```

QUESTION 9 — PREDICT EACH LINE'S OUTPUT

Run in order. Some lines print nothing but `+`, and one prints **three** things.

LINE	OUTPUT
`Tracer a("A");`	
`Tracer b = a;`	
`Tracer c = std::move(a);`	
`Tracer d = MakeTracer();`	
`v.push_back(b);`	
`v.push_back(std::move(b));`	
`v.push_back(Tracer("X"));`	
`v.emplace_back("Y");`	

QUESTION 10 — SHORT ANSWER

1. `Tracer d = MakeTracer();` printed no copy and no move. What did the compiler do instead?

   _____

2. What's the difference between `push_back(Tracer("X"))` and `emplace_back("Y")`?

   _____

3. After line 3, what is `a`'s name? _____ Is `a` still a valid object? _____

Object Lifetime & Ownership — Who Cleans Up?

Review worksheet — destructors, the Rule of Three / Five / Zero, move semantics, RAII, smart pointers, ownership — builds on Worksheets 03, 04 & 06

part 5 The Rule of Five, and the Rule of Zero

RULE	YOU WRITE	WHEN
Three	destructor, copy constructor, copy assignment	Your class directly manages a resource (memory, a file, a lock)
Five	the three, plus move constructor and move assignment	Same, and you also want cheap moves
Zero	*none of them*	Your members already manage themselves (`string`, `vector`, `unique_ptr`, ...). **This should be most of your classes.**

A trap: if you declare *any* of the destructor, copy constructor, or copy assignment, the compiler stops generating the move operations. A "move" then quietly becomes a copy. `Spy` below prints `copy` or `move` when it is copied or moved.

```
struct Plain {
    Spy s;
};
struct WithDtor {
    Spy s;
    ~WithDtor() {
    }
};
struct WithCopy {
    Spy s;
    WithCopy() = default;
    WithCopy(const WithCopy& other)
        : s(other.s) {
    }
};
```

QUESTION 11 — COPY OR MOVE?

`Plain p1; Plain p2 = std::move(p1);` prints _____

`WithDtor w1; WithDtor w2 = std::move(w1);` prints _____

`WithCopy c1; WithCopy c2 = std::move(c1);` prints _____

That empty `~WithDtor() {}` does nothing useful. What does it cost?

_____

QUESTION 12 — RULE OF ZERO — REWRITE INTSTACK

Rewrite `IntStack` (constructor, `Push`, `Pop`, `Top`, `Size`) so it needs **none** of the five special members, and still copies and moves correctly.

Who now does the work your hand-written destructor, copy, and move used to do?

_____

Object Lifetime & Ownership — Who Cleans Up?

Review worksheet — destructors, the Rule of Three / Five / Zero, move semantics, RAII, smart pointers, ownership — builds on Worksheets 03, 04 & 06

part 5 The Rule of Five, and the Rule of Zero (continued)

Some things should never be copied: an open file, a network connection, a game window. Two copies would both try to close the same thing. Use `= delete` to forbid the operation. (`= default` asks for the compiler's version on purpose.)

```
class FileHandle {
public:
    explicit FileHandle(const char* path)
        : file_(fopen(path, "w")) {
    }
    ~FileHandle() {
        if (file_ != nullptr) {
            fclose(file_);
        }
    }
    FileHandle(const FileHandle&) = delete;
    FileHandle& operator=(const FileHandle&) = delete;
    bool IsOpen() const {
        return file_ != nullptr;
    }
private:
    FILE* file_;
};
```

QUESTION 13 — COMPILES? Y / N

CODE	Y / N	WHY?
`FileHandle log("game.log");`		
`FileHandle copy = log;`		
`void Save(FileHandle f);` then `Save(log);`		
`void Save(FileHandle& f);` then `Save(log);`		
`vector<FileHandle> logs;` then `logs.push_back(log);`		

QUESTION 14 — CLASSIFY EACH CLASS

Rule of **Zero**, **Three/Five**, or **copying deleted**? One word or number each.

`Card` with an `int value_` _____

`Hand` with a `vector<Card> cards_` _____

`Party` with a `vector<unique_ptr<Character>>` (copying a party makes no sense) _____

`Matrix` with `double* cells_ = new double[rows * cols]` _____

`Player` with a `string name_` and a `Hand hand_` _____

Object Lifetime & Ownership — Who Cleans Up?

Review worksheet — destructors, the Rule of Three / Five / Zero, move semantics, RAII, smart pointers, ownership — builds on Worksheets 03, 04 & 06

part 6 RAII: Let the Destructor Clean Up

RAII ("Resource Acquisition Is Initialization") is the most important C++ idiom: *tie every resource to an object's lifetime*. Acquire it in the constructor and release it in the destructor. Destructors of local objects always run when a scope ends, whether by reaching `}`, by `return`, or by an **exception**. So the resource is always released. `FileHandle`, `vector`, `string`, and `unique_ptr` are all RAII.

```
void Risky(bool fail) {
    int* scores = new int[100];
    Tracer guard("guard");
    if (fail) {
        throw runtime_error("boom");
    }
    delete[] scores;
    cout << "done ";
}

int main() {
    try {
        Risky(false);
        Risky(true);
        cout << "after ";
    } catch (const exception& e) {
        cout << "caught:" << e.what() << " ";
    }
}
```

QUESTION 15 — EXCEPTIONS AND DESTRUCTORS

1. Write the output:

   _____

2. Did `guard`'s destructor run when the exception was thrown? _____ What leaked on the second call?

   _____

3. Change **one line** so nothing can leak, with no `delete` and no `try`. Write the new line:

   _____

QUESTION 16 — WRITE AN RAII CLASS — LOGSCOPE

Write `LogScope`. Its constructor takes a name and prints `enter NAME`, and its destructor prints `exit NAME`. Copying should be impossible.

```
void Inner() {
    LogScope s("inner");
    throw runtime_error("fail");
}
void Outer() {
    LogScope s("outer");
    Inner();
    cout << "unreachable ";
}
```

`main` runs `try { Outer(); } catch (...) { cout << "caught"; }`. Output:

_____

_____

Object Lifetime & Ownership — Who Cleans Up?

Review worksheet — destructors, the Rule of Three / Five / Zero, move semantics, RAII, smart pointers, ownership — builds on Worksheets 03, 04 & 06

part 7 Smart Pointers

builds on W06's `unique_ptr<Character>`

Smart pointers are RAII for heap objects. Each one says **who owns** the object:

TYPE	OWNERSHIP	COPY?	OBJECT DELETED WHEN…
`unique_ptr<T>`	exactly one owner	No. Move only.	the owner is destroyed or reset
`shared_ptr<T>`	shared by several owners, with a count	Yes. Each copy adds 1 to the count.	the **last** owner goes away (count reaches 0)
`weak_ptr<T>`	not an owner; it only watches a `shared_ptr`	Yes	never by this pointer (it can check `expired()`)

QUESTION 17 — PREDICT THE OUTPUT

```
auto a = make_unique<Tracer>("U");
auto b = std::move(a);
cout << (a == nullptr);          // (a)
// auto c = b;                    // (b)

auto s1 = make_shared<Tracer>("S");
cout << s1.use_count();          // (c)
{
    auto s2 = s1;
    cout << s1.use_count();      // (d)
}
cout << s1.use_count();          // (e)
weak_ptr<Tracer> w = s1;
cout << s1.use_count();          // (f)
cout << w.expired();             // (g)
s1.reset();                      // (h)
cout << w.expired();             // (i)
```

(a) _____

(b) compiles? _____

(c) _____

(d) _____

(e) _____

(f) _____

(g) _____

(h) prints? _____

(i) _____

(`bool` prints as `1` / `0`.) When is the `"U"` Tracer destroyed? _____

QUESTION 18 — THE CYCLE

```
class Team {
    vector<shared_ptr<Player>> roster_;
    // ~Team() prints "~Team"
};

class Player {
    shared_ptr<Team> team_;
    // ~Player() prints "~Player"
};

{
    auto mustangs = make_shared<Team>("Mustangs");
    auto rhea = make_shared<Player>("Rhea");
    mustangs->Add(rhea);       // roster_ holds rhea
    rhea->Join(mustangs);      // team_ holds mustangs
}
cout << "end";
```

1. Just before the `}`, what are `mustangs.use_count()` and `rhea.use_count()`? _____
2. What prints? _____ Why?

_____

3. Change one member's type to fix it. Which one, and to what?

_____

Object Lifetime & Ownership — Who Cleans Up?

Review worksheet — destructors, the Rule of Three / Five / Zero, move semantics, RAII, smart pointers, ownership — builds on Worksheets 03, 04 & 06

part 8 Ownership Is a Design Decision

applies to P01

A function's signature tells the caller who will own the object afterward. Good code makes ownership obvious from the types alone. The default is: **hold by value, and borrow by reference**. Reach for a pointer only when you need what a pointer adds.

QUESTION 19 — WHAT DOES THE SIGNATURE PROMISE?

Match each signature to a meaning (A–F).

`void Inspect(const Card& c)`	_____
`void Discard(unique_ptr<Card> c)`	_____
`void Track(shared_ptr<Card> c)`	_____
`void MaybeShow(const Card* c)`	_____
`unique_ptr<Card> Draw()`	_____
`const Card& TopCard() const`	_____

A. Takes ownership. The caller gives it up.
B. Borrows, read-only, never null.
C. Gives ownership to the caller.
D. Becomes one of several owners.
E. Borrows, read-only, and it might be null.
F. Lets the caller look at something *this* object owns. Don't keep the reference longer than the object lives.

QUESTION 20 — PICK THE MEMBER TYPE

Choose: **value**, **reference**, **`unique_ptr`**, **`shared_ptr`**, or **`weak_ptr`**.

A `Hand` holds its cards _____

`main()` holds the one `GameView` chosen at run time (Console or TUI) _____

A `ConsoleView` needs the game it displays, which `main()` owns and which outlives the view _____

Several sprites draw the same texture. Free it when the last sprite is gone. _____

A `Player` remembers its `Team`, but the team owns the players _____

A `Party` holds Warriors and Wizards through their base class _____

QUESTION 21 — SHORT ANSWER

Why is `vector<Card>` a better choice for a `Hand` than `vector<unique_ptr<Card>>`, but `vector<unique_ptr<Character>>` is the right choice for a `Party`? (Hint: W06, slicing.)

_____

_____

Object Lifetime & Ownership — Who Cleans Up?

Review worksheet — destructors, the Rule of Three / Five / Zero, move semantics, RAII, smart pointers, ownership — builds on Worksheets 03, 04 & 06

part 9 Find the Error

all four compile

CASE A — RUNS, BUT…

```
const string& Greeting() {
    string message = "Hello";
    return message;
}

cout << Greeting();
```

What's wrong, and what's the simplest fix?

CASE B — RUNS, BUT…

```
vector<string> names = {"Thorin"};
string& first = names[0];
names.push_back("Rhea");
names.push_back("Vex");
cout << first;
```

Why might `first` no longer refer to a real string?

CASE C — CRASHES

```
int* raw = new int(5);
unique_ptr<int> p(raw);
unique_ptr<int> q(raw);
```

What happens at the end of the scope? What habit prevents it?

CASE D — RUNS, BUT…

```
string name = "Malakai";
vector<string> party;
party.push_back(std::move(name));
cout << "Welcome, " << name;
```

What does it print, and is that guaranteed?

TAKEAWAY — THREE RULES

1. Every resource has exactly one clear owner, and that owner is an object whose destructor releases it. **2.** Prefer the Rule of Zero: build classes out of members that already manage themselves. **3.** A reference or raw pointer *borrows*. It must never outlive what it points to.

Object Lifetime & Ownership — Who Cleans Up?

Review worksheet — destructors, the Rule of Three / Five / Zero, move semantics, RAII, smart pointers, ownership — builds on Worksheets 03, 04 & 06

cheat sheet Lifetime and Ownership at a Glance

CONCEPT	TYPICAL SYNTAX	PURPOSE
Destructor	`~IntStack()`	Release what the object owns. Runs automatically when its lifetime ends.
Copy constructor	`IntStack(const IntStack& other)`	Build a **new** object as an independent (deep) copy
Copy assignment	`IntStack& operator=(const IntStack& other)`	Overwrite an **existing** object: check self-assignment, free the old data, copy, `return *this`
Move constructor	`IntStack(IntStack&& other) noexcept`	Steal resources from a temporary, then leave the source empty but destructible
Move assignment	`IntStack& operator=(IntStack&& other) noexcept`	Free your own data, steal the other's, empty the source
`std::move`	`v.push_back(std::move(name));`	A cast that says “I'm done with this, you may steal it.” It doesn't move anything itself.
`= delete` / `= default`	`FileHandle(const FileHandle&) = delete;`	Forbid an operation, or ask for the compiler's version on purpose
Rule of Three / Five	—	Managing a raw resource yourself? Write all three (or five).
Rule of Zero	`vector<int> data_;`	Let members manage themselves and write none. **The default choice.**
RAII	`FileHandle f("x.txt");`	Acquire in the constructor, release in the destructor. Safe even when an exception is thrown.
`unique_ptr`	`auto p = make_unique<Wizard>("Merlin");`	One owner, move-only. Deletes automatically.
`shared_ptr`	`auto t = make_shared<Texture>("orc.png");`	Several owners. Deleted when the count reaches 0.
`weak_ptr`	`weak_ptr<Team> team_;`	Watches without owning. Breaks `shared_ptr` cycles.

decision Which One Should This Member Be?

```
Does this object OWN the thing?
 ├─ no  ─► reference (always there)  ·  raw pointer (might be null)  ·  weak_ptr (shared thing that may die)
 └─ yes ─► Must it be polymorphic, or does it outlive the scope that made it?
           ├─ no  ─► a plain VALUE member: Card, string, vector<Card>    (Rule of Zero)
           └─ yes ─► Is there exactly one owner?
                     ├─ yes ─► unique_ptr<T>          (the usual answer)
                     └─ no  ─► shared_ptr<T>          (only when ownership really is shared)
```